PCCCS495 — Term II Project Report

Real-Time Chat Application

with Observer-Based Messaging

Multithreading + Observer Pattern | Java | OOP Course Project

Name	Rwitabrato Hanpal
Enrollment No.	12024052017027
Roll No.	10
Department	CSE(IoT,CSBT)
Course	PCCCS495 — Advanced Programming (OOPS)
Topic	Observer Pattern + Multithreading
Institution	Institute of Engineering & Management, Salt Lake

Table of Contents

1. Introduction
2. Problem Statement
3. OOP Concepts Applied
4. System Architecture
5. Thread Safety Strategy
6. Core Features
7. Class Structure
8. Testing Strategy
9. How to Run
10. Scalability & Future Direction
11. Conclusion

1. Introduction

This report documents the design, implementation, and testing of a Real-Time Chat Application built in Java as part of the PCCCS495 Term II project. The application demonstrates a thorough application of object-oriented principles by implementing a multi-client messaging system where each connected user runs on a dedicated thread and receives messages through the Observer Pattern.

The project goes beyond a basic tutorial by incorporating thread-safe data structures, an ExecutorService thread pool, persistent chat logging, a modern Swing GUI client, a console client, and a complete test suite with unit and integration tests — reflecting industry-grade OOP practice.

2. Problem Statement

Most introductory-level chat programs handle only a single client at a time and rely on polling loops, which obscures the event-driven logic underneath. This makes it hard to reason about concurrency or understand how one action propagates to multiple recipients. The proposed Real-Time Chat Application addresses this by building a multi-client server where each connected user runs on a dedicated thread and receives messages through the Observer Pattern. Every inbound message triggers a broadcast through a registered observer list, demonstrating event propagation, thread lifecycle management, and clean separation between connection handling and notification delivery.

Target Users

- Students collaborating on group projects who need a lightweight, self-hosted messaging tool
- Small development teams requiring internal chat without third-party service dependencies
- Educators seeking a live classroom demo of threading and the Observer Pattern in action

3. OOP Concepts Applied

Concept	How It Is Applied	Location
Abstraction	ChatObserver interface defines update(message) — server depends only on this contract, never on concrete types	ChatObserver.jav a

Inheritance	ClientHandler extends Thread, overriding run() for per-client socket processing	ClientHandler.jav a
Polymorphism	broadcast() calls update() identically on socket clients, file loggers, and UI panels	ChatServer.java
Encapsulation	ChatServer keeps observer list and socket logic private; exposes only register(), broadcast(), etc.	ChatServer.java
Exception Handling	SocketException, IOException, ClientDisconnectException caught at separate layers	ClientHandler.jav a
Collections	CopyOnWriteArrayList for lock-free observer reads; LinkedBlockingQueue for message queuing	ChatServer.java
Threads	ExecutorService.newFixedThreadPool(50) manages client threads; each ClientHandler is a Thread	ChatServer.java
Design Pattern	Observer Pattern — ChatServer is Subject; ClientHandler and MessageLogger are registered Observers	All core files

4. System Architecture

The system follows a layered, event-driven architecture built around the Subject–Observer relationship. A ChatObserver interface defines the notification contract. ChatServer (Subject) owns the registered observer list and drives all broadcast logic. Each client connection is wrapped in a ClientHandler that extends Thread and implements ChatObserver. A MessageLogger persists every message to disk without coupling to client code, while PersistenceManager isolates all file I/O from the server layer.

```
ChatObserver (Interface)
|
+-- ClientHandler (Thread + Observer) one per connected client
+-- MessageLogger (Observer) writes every message to file
|
+-- PersistenceManager isolated file I/O layer
ChatServer (Subject)
+-- CopyOnWriteArrayList thread-safe observer registry
+-- LinkedBlockingQueue thread-safe message queue
+-- ExecutorService (pool, max 50) manages all client threads
Client Layer
+-- SwingClient (dark-theme GUI, typing indicator, DM bubbles)
+-- ConsoleClient (terminal-based, dedicated listener thread)
```

Message Flow

```
Client types message
-> ClientHandler.listenForMessages() reads from socket
-> server.broadcast(message) called
-> message placed into LinkedBlockingQueue
-> processQueue() drains queue
-> observer.update(message) called on every ChatObserver
| |
ClientHandler MessageLogger
writes to socket writes to log file
```

5. Thread Safety Strategy

Because multiple ClientHandler threads run concurrently and all share the observer list and message queue on ChatServer, thread safety is a critical concern. The following strategy prevents race conditions across all shared resources:

Shared Resource	Solution	Reason
Observer list	CopyOnWriteArrayList	Lock-free reads; safe iteration while clients connect/disconnect
Message queue	LinkedBlockingQueue	Thread-safe producer/consumer; blocks cleanly under pressure
Typing users set	ConcurrentHashMap.newKeySet()	Concurrent add/remove without synchronization overhead
Client threads	ExecutorService (fixed pool)	Caps at 50 threads; prevents unbounded resource consumption
Socket I/O per client	Dedicated thread per handler	Each ClientHandler owns its socket exclusively — no sharing

6. Core Features

- **Multi-client connectivity:** Multiple users connect simultaneously over a local network via TCP sockets
- **Real-time broadcast:** Every message is instantly forwarded to all registered observers via Observer Pattern
- **Username registration:** Each client registers a username on connect; join/leave events are broadcast
- **Private direct messaging:** /msg routes a message to one specific client only
- **Typing indicator:** Server broadcasts [TYPING] signals; clients show 'X is typing...' in real time
- **Persistent chat log:** MessageLogger writes every broadcast to logs/chat_history.txt with timestamps
- **Swing GUI client:** Dark-theme interface with chat bubbles, sidebar, DM rendering, and status bar
- **Console client:** Terminal-based client with a dedicated daemon listener thread
- **Graceful disconnect handling:** ClientDisconnectException ensures clean unregistration on unexpected exits
- **Shutdown hook:** Runtime.addShutdownHook() cleanly stops the thread pool on Ctrl+C

7. Class Structure

Class / File	Package	Role
ChatObserver	observer	Interface — defines update(message) contract
ChatServer	server	Subject — observer registry, broadcast, thread pool
ClientHandler	client	Thread + Observer — handles one socket connection
ConsoleClient	client	Terminal-based chat client
SwingClient	client	Dark-theme GUI chat client
MessageLogger	persistence	Observer — writes all messages to file
PersistenceManager	persistence	Handles all file I/O for chat history
ClientDisconnectExc eptio n	exception	Custom exception for unexpected client exits
Main	root	Entry point — bootstraps ChatServer
MockObserver	test	Test double — records update() calls for assertions
ChatServerTest	test	Unit tests for observer registration and broadcast
IntegrationTest	test	Full pipeline tests with live sockets

8. Testing Strategy

Unit Tests — ChatServerTest.java

Unit tests verify the core observer mechanics of ChatServer in complete isolation — no network, no sockets, no threads required. MockObserver acts as a test double implementing ChatObserver, recording every update() call for assertion.

Test	Verifies
testRegisterObserver	Observer receives broadcast after registering
testBroadcastReachesAll	All registered observers receive the same message
testUnregisterObserver	Observer does NOT receive messages after unregistering

testBroadcastAfterUnregister	Remaining observers still receive; unregistered one does not
testMultipleMessages	All 3 queued messages arrive in order
testEmptyBroadcast	Empty string broadcast still reaches observers

Integration Tests — IntegrationTest.java

Integration tests spin up a live ChatServer on port 5050 and connect real socket clients to verify the full end-to-end pipeline.

Test	Verifies
testClientConnectsAndReceivesBroadcast	Alice sends a message — Bob receives it via socket
testPrivateMessageReachesOnlyTarget	DM to Bob arrives at Bob; Charlie does NOT receive it
testClientDisconnectNotifiesOthers	Alice quits — Bob sees the leave notification

9. How to Run

Prerequisites

- JDK 17 or higher — verify with: java -version
- Terminal or Command Prompt

Step 1 — Compile

```
cd src
find . -name '*.java' | xargs javac
```

Step 2 — Start the Server

```
java com.chatapp.Main
# Expected output:
# Server started on port 5000
# Log file created at: logs/chat_history.txt
```

Step 3 — Connect Clients

```
# GUI client (open a new terminal):
java com.chatapp.client.SwingClient
# Console client (open another terminal):
java com.chatapp.client.ConsoleClient
```

Step 4 — Chat Commands

Command	Description
(any text)	Send a message to all connected users
/msg	Send a private direct message to a specific user
/quit	Gracefully disconnect from the server

Step 5 — Run Tests

```
# Unit tests:
java com.chatapp.test.ChatServerTest
# Integration tests:
java com.chatapp.test.IntegrationTest
```

10. Scalability & Future Direction

The current thread-per-client model managed via ExecutorService works well up to approximately 50 concurrent users — each connection gets its own thread from the fixed pool, capping resource usage and preventing unbounded thread spawning.

For higher user counts, Java NIO with Selector and SocketChannel would allow a single thread to manage thousands of connections via non-blocking I/O — polling for ready channels instead of blocking on each one. This is the architecture used by production servers like Netty and is noted as the natural next step for this system.

Approach	Concurrency Model	Suitable For
Current — Thread Pool	ExecutorService.newFixedThread Pool(5 0)	Up to 50 concurrent users
Future — Java NIO	Selector + SocketChannel (non-blocking)	Thousands of concurrent users
Future — Virtual Threads	Project Loom — JDK 21+ virtual threads	High concurrency, low memory

11. Conclusion

The Real-Time Chat Application successfully demonstrates all eight required OOP concepts in a system that mirrors how production chat software actually works. The Observer Pattern is used as the backbone of the notification system rather than a superficial addition — every message flows through the same interface regardless of whether it goes to a socket, a file, or a UI panel. Thread safety is addressed systematically with the right Java concurrent utilities at each layer. The project was developed incrementally with disciplined commits, fully tested with both unit and integration tests, and documented with architecture diagrams, a UML class diagram, and this final report.

The architecture is intentionally extensible — adding a new observer type (a moderation bot, a metrics collector, a push notification service) requires only implementing the ChatObserver interface and registering it with ChatServer. No existing classes need modification. This demonstrates the Open/Closed Principle in practice and reflects the educational value of design-first object-oriented thinking.